

UNIT 4

Query Processing

Query Processing is the activity performed in extracting data from the database. In query processing, it takes various steps for fetching the data from the database. The steps involved are:

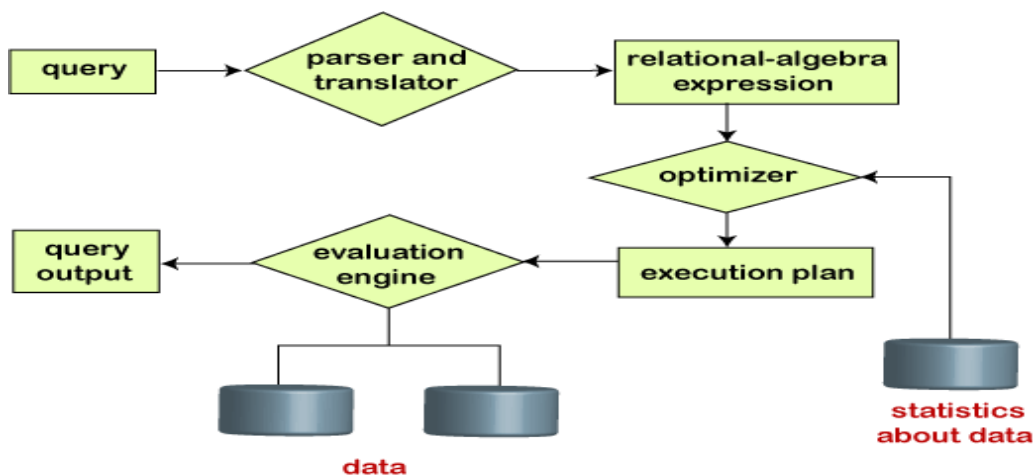
1. Parsing and translation
2. Optimization
3. Evaluation

The query processing works in the following way:

Parsing and Translation

As query processing includes certain activities for data retrieval. Initially, the given user queries get translated in high-level database languages such as SQL. It gets translated into expressions that can be further used at the physical level of the file system. After this, the actual evaluation of the queries and a variety of query -optimizing transformations and takes place. Thus before processing a query, a computer system needs to translate the query into a human-readable and understandable language. Consequently, SQL or Structured Query Language is the best suitable choice for humans. But, it is not perfectly suitable for the internal representation of the query to the system. Relational algebra is well suited for the internal representation of a query. The translation process in query processing is similar to the parser of a query. When a user executes any query, for generating the internal form of the query, the parser in the system checks the syntax of the query, verifies the name of the relation in the database, the tuple, and finally the required attribute value. The parser creates a tree of the query, known as 'parse-tree.' Further, translate it into the form of relational algebra. With this, it evenly replaces all the use of the views when used in the query.

Thus, we can understand the working of a query processing in the below-described diagram:



Steps in query processing

Suppose a user executes a query. As we have learned that there are various methods of extracting the data from the database. In SQL, a user wants to fetch the records of the employees whose salary is greater than or equal to 10000. For doing this, the following query is undertaken:

select emp_name from Employee where salary>10000;

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{\text{salary} > 10000} (\pi_{\text{salary}} (\text{Employee}))$
- $\pi_{\text{salary}} (\sigma_{\text{salary} > 10000} (\text{Employee}))$

After translating the given query, we can execute each relational algebra operation by using different algorithms. So, in this way, a query processing begins its working.

Evaluation

For this, with addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus, after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.
- Such relational algebra with annotations is referred to as **Evaluation Primitives**. The evaluation primitives carry the instructions needed for the evaluation of the operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also referred to as **the query execution plan**.
- A **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

Optimization

- The cost of the query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as Query Optimization.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because the overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of the query.

Estimating Query Cost

In the previous section, we understood about Query processing steps and evaluation plan. Though a system can create multiple plans for a query, the chosen method should be the best of all. It can be done by comparing each possible plan in terms of their estimated cost. For calculating the net estimated cost of any plan, the cost of each operation within a plan should be determined and combined to get the net estimated cost of the query evaluation plan.

The cost estimation of a query evaluation plan is calculated in terms of various resources that include:

- Number of disk accesses
- Execution time taken by the CPU to execute a query
- Communication costs in distributed or parallel database systems.

To estimate the cost of a query evaluation plan, we use the number of blocks transferred from the disk, and the number of disks seeks. Suppose the disk has an average block access time of t_s seconds and takes an average of t_T seconds to transfer x data blocks. The block access time is the sum of disk seeks time and rotational latency. It performs S seeks than the time taken will be $b \cdot t_T + S \cdot t_s$ seconds. If $t_T = 0.1$ ms, $t_s = 4$ ms, the block size is 4 KB, and its transfer rate is 40 MB per second. With this, we can easily calculate the estimated cost of the given query evaluation plan.

Generally, for estimating the cost, we consider the worst case that could happen. The users assume that initially, the data is read from the disk only. But there must be a chance that the information is already present in the main memory. However, the users usually ignore this effect, and due to this, the actual cost of execution comes out less than the estimated value.

The response time, i.e., the time required to execute the plan, could be used for estimating the cost of the query evaluation plan. But due to the following reasons, it becomes difficult to calculate the response time without actually executing the query evaluation plan:

- When the query begins its execution, the response time becomes dependent on the contents stored in the buffer. But this information is difficult to retrieve when the query is in optimized mode, or it is not available also.
- When a system with multiple disks is present, the response time depends on an interrogation that in "what way accesses are distributed among the disks?". It is difficult to estimate without having detailed knowledge of the data layout present over the disk.
- Consequently, instead of minimizing the response time for any query evaluation plan, the optimizers find it better to reduce the total **resource consumption** of the query plan. Thus to estimate the cost of a query evaluation plan, it is good to minimize the resources used for accessing the disk or use of the extra resources.

Join processing algorithms

Nested-Loop Join Algorithm

In our previous section, we learned about joins and various types of joins. In this section, we will know about the nested loop join algorithm.

A nested loop join is a join that contains a pair of nested for loops. To perform the nested loop join i.e., θ on two relations r and s , we use an algorithm known as the **Nested loop join algorithm**. The computation takes place as:

$r \bowtie_{\theta} s$

where r is known as the **outer relation** and s is the **inner relation** of the join. It is because the for loop of r encloses the for loop of s .

Nested-Loop Join Algorithm

Let's discuss the algorithm of nested-loop join:

```
for each tuple  $t_r$  in  $r$  do begin
    for each tuple  $t_s$  in  $s$  do begin
        test pair  $(t_r, t_s)$  to test if they satisfy the given join condition ??
        if test satisfied
            add  $t_r \cdot t_s$  to the result;
    end inner loop
end outer loop
```

In the algorithm, t_r and t_s are the tuples of relations r and s , respectively. The notation $t_r \cdot t_s$ is a tuple constructed by concatenating the attribute values of tuples t_r and t_s .

With the help of the algorithm, we understood the following points:

- The nested-loop join does not need any indexing similar to a linear file scan for accessing the data.
- Nested-loop join does not care about the given join condition. It is suitable for each given join condition.
- The nested-loop join algorithm is expensive in nature. It is because it computes and examines each pair of tuples in the given two relations.

Cost Analysis of Nested-loop Join Algorithm

For analyzing the cost of the nested-loop join algorithm, consider a number of pairs of tuples as $n_r * n_s$. Here, n_r specifies the number of tuples in relation r and n_s specifies the number of tuples in relation s . For computing the cost, perform a complete scan on relation s . Thus,

Total number of block transfers in worst case = $n_r * b_s + b_r$

Total number of seeks required in worst case = $n_r + b_r$

Here, b_s and b_r are the number of blocks holding the tuples of relation r and s , respectively.

In the best case, both relations r and s have sufficient memory to fit in the memory simultaneously. So, each block will read only once. Thus,

Total number of block transfers in best case = $b_r + b_s$. Total

number of seeks required = $2(nr + br)$

In case if any one of the relations given fits entirely into the memory, it is a must to use that relation as the inner relation. It is because we will read the inner relation only once. Thus,

Total number of block transfers in such case = $b_r + b_s$

Total number of seeks required = $2(nr + br)$

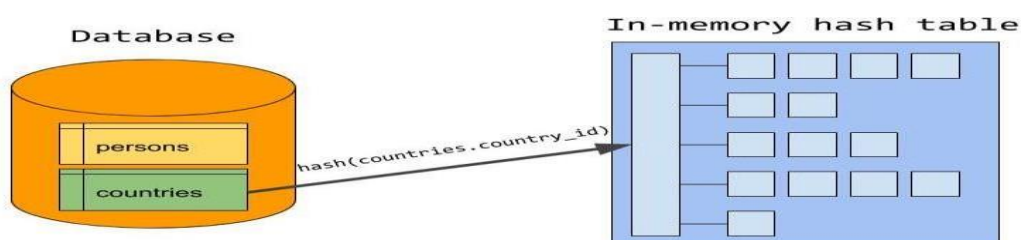
Hash join

Hash join is a way of executing a join where a hash table is used to find matching rows between the two inputs (an input is one or more tables). It is typically more efficient than nested loop joins, especially if one of the inputs can fit in memory. To see how it works, we will use the following query as an example:

```
1 SELECT
2   given_name,      country_name
3 FROM
4   persons JOIN countries ON persons.country_id = countries.country_id;
```

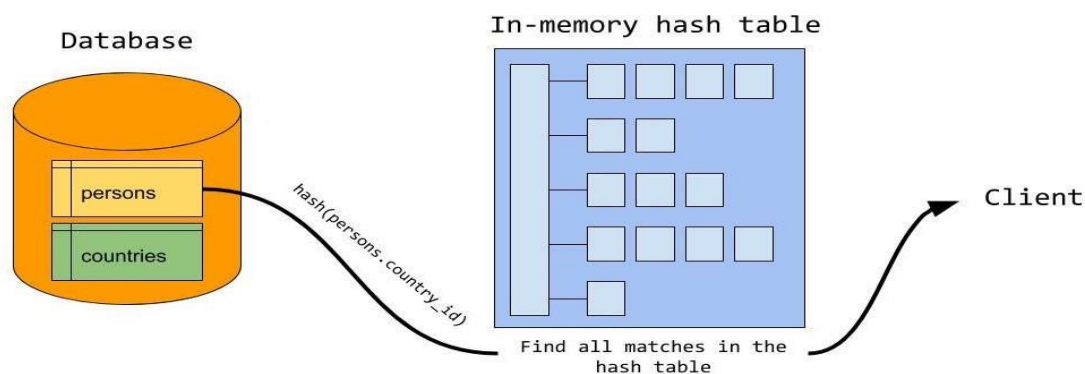
The build phase

The literature usually divides hash join in two phases; the build phase and the probe phase. In the build phase, the server builds an in-memory hash table where rows from one of the inputs are stored, using the join attribute(s) as the hash table key. This input is also known as the build input, and let us assume that '*countries*' is designated as the build input. Ideally, the server will choose the smaller of the two inputs as the build input (measured in bytes, not number of rows). Since '*countries.country_id*' is the join condition that belongs to the build input, this is used as the key in the hash table. Once all the rows has been stored in the hash table, the build phase is done.



The probe phase

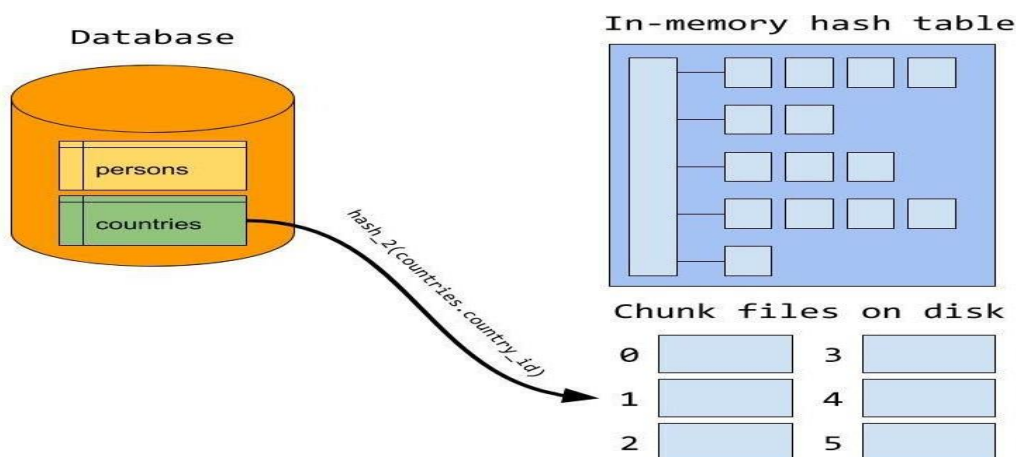
During the probe phase, the server starts reading rows from the probe input (*persons* in our example). For each row, the server probes the hash table for matching rows using the value from *persons.country_id* as the lookup key. For each match, a joined row is sent to the client. In the end, the server has scanned each input only once, using constant time lookup to find matching rows between the two inputs.



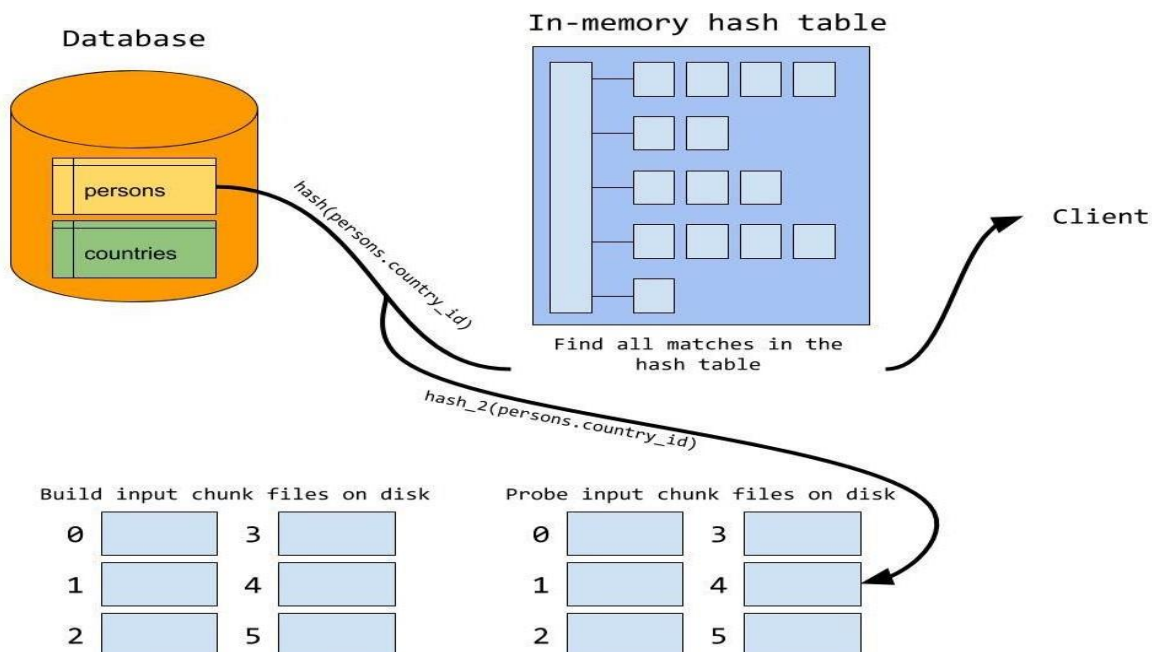
This works very well given that the server can store the entire build input in memory. The amount of memory available is controlled by the system variable `join_buffer_size`, and can be adjusted at runtime. But what happens if the build input is bigger than the available memory? We spill to disk!

Spill to disk

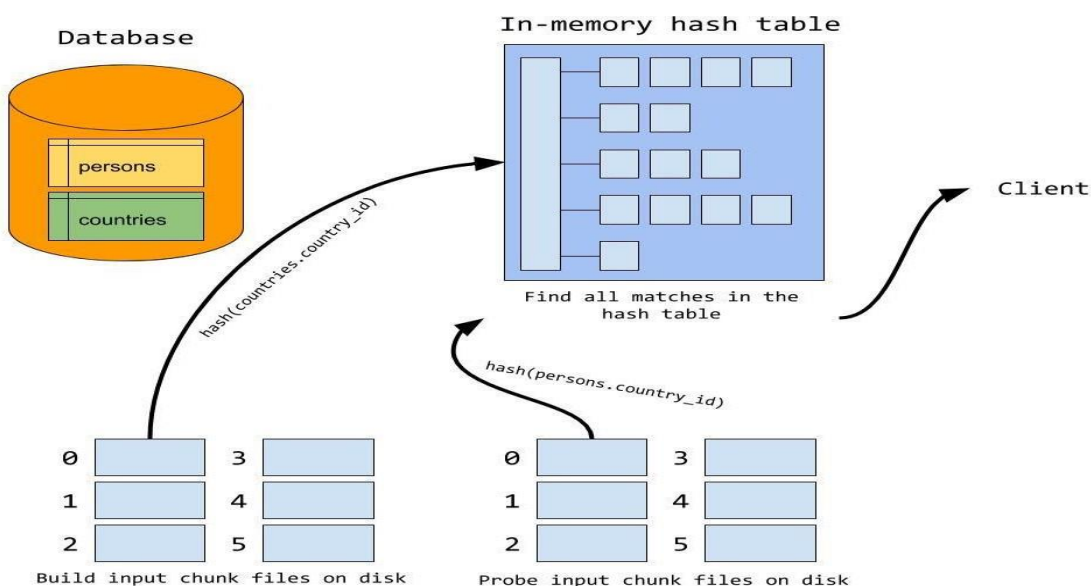
When the memory goes full during the build phase, the server writes the rest of the build input out to a number of chunk files on disk. The server tries to set the number of chunks so that the largest chunk fits exactly in memory (we will see why soon), but MySQL has a hard upper limit of 128 chunk files per input. Which chunk file a row is written to is determined by calculating a hash of the join attribute. Note that in the illustration, there is a different hash function used than the one used in the in-memory build phase. We will see why a bit later.



During the probe phase, the server probes for matching rows in the hash table, just like when everything fits in memory. But in addition, a row may also match one of the rows from the build input that is written to disk. Thus, each row from the probe input is also written to a set of chunk files. Which chunk file a row is written to is determined using the same hash function and formula that is used when the build input is written to disk. This way, we know for sure that matching rows between the two inputs will be located in the same pair of chunk files.



When the probe phase is done, we start reading chunk files from disk. In general, the server does a build and probe phase using the first set of chunk files as the build and probe input. We load the first chunk file from the build input into the in-memory hash table. This explains why we want the largest chunk to fit exactly in memory; if the chunk files are too big we need to split it up in even smaller chunks. After the build chunk is loaded, we read the corresponding chunk file from the probe input and probe for matches in the hash table, just like when everything fits in memory. When the first pair of chunk files is processed, we move to the next pair of chunk files, continuing until all pair of chunk files has been processed.



You might have guessed now why we should use two different hash functions when

partitioning rows out to chunk files and writing rows to the hash table. If we were to use the same hash function for both operations, we would get an extremely bad hash table when loading a build chunk file into the hash table since all rows in the same chunk file would hash to the same value.

Query Optimization

Query optimization is of great importance for the performance of a relational database, especially for the execution of complex SQL statements. A query optimizer decides the best methods for implementing each query.

The query optimizer selects, for instance, whether or not to use indexes for a given query, and which join methods to use when joining multiple tables. These decisions have a tremendous effect on SQL performance, and query optimization is a key technology for every application, from operational Systems to data warehouse and analytical systems to content management systems.

There is the various principle of Query Optimization are as follows –

- **Understand how your database is executing your query** – The first phase of query optimization is understanding what the database is performing. Different databases have different commands for this. For example, in MySQL, one can use the "EXPLAIN [SQL Query]" keyword to see the query plan. In Oracle, one can use the "EXPLAIN PLAN FOR [SQL Query]" to see the query plan.
- **Retrieve as little data as possible** – The more information restored from the query, the more resources the database is required to expand to process and save these records. For example, if it can only require to fetch one column from a table, do not use 'SELECT *'.
- **Store intermediate results** – Sometimes logic for a query can be quite complex. It is possible to produce the desired outcomes through the use of subqueries, inline views, and UNION-type statements. For those methods, the transitional results are not saved in the database but are directly used within the query. This can lead to achievement issues, particularly when the transitional results have a huge number of rows.

There are various query optimization strategies are as follows –

- **Use Index** – It can be using an index is the first strategy one should use to speed up a query.
- **Aggregate Table** – It can be used to pre-populating tables at higher levels so less amount of information is required to be parsed.
- **Vertical Partitioning** – It can be used to partition the table by columns. This method reduces the amount of information a SQL query required to process.
- **Horizontal Partitioning** – It can be used to partition the table by data value, most often time. This method reduces the amount of information a SQL query required to process.
- **De-normalization** – The process of de-normalization combines multiple tables into a single table. This speeds up query implementation because fewer table joins are required.
- **Server Tuning** – Each server has its parameters and provides tuning server parameters so that it can completely take benefit of the hardware resources that can significantly speed up query implementation.

Transforming Relational Expressions

The first step of the optimizer says to implement such expressions that are logically equivalent to the given expression. For implementing such a step, we use the equivalence rule that describes the method to transform the generated expression into a logically equivalent expression.

Although there are different ways through which we can express a query, with different costs. But for expressing a query in an efficient manner, we will learn to create alternative as well as equivalent expressions of the given expression, instead of working with the given expression. Two relational-algebra expressions are equivalent if both the expressions produce the same set of tuples on each legal database instance. A **legal database instance** refers to that database system which satisfies all the integrity constraints specified in the database schema. However, the sequence of the generated tuples may vary in both expressions, but they are considered equivalent until they produce the same tuples set.

Equivalence Rules

The equivalence rule says that expressions of two forms are the same or equivalent because both expressions produce the same outputs on any legal database instance. It means that we can possibly replace the expression of the first form with that of the second form and replace the expression of the second form with an expression of the first form. Thus, the optimizer of the query-evaluation plan uses such an equivalence rule or method for transforming expressions into the logically equivalent one.

The optimizer uses various equivalence rules on relational-algebra expressions for transforming the relational expressions. For describing each rule, we will use the following symbols:

θ , θ_1 , θ_2 ... : Used for denoting the predicates.

L_1 , L_2 , L_3 ... : Used for denoting the list of attributes.

E , E_1 , E_2 : Represents the relational-algebra expressions.

Let's discuss a number of equivalence rules:

Rule 1: Cascade of σ

This rule states the deconstruction of the conjunctive selection operations into a sequence of individual selections. Such a transformation is known as a **cascade of σ** .

$$\sigma_{\theta_1 \wedge \theta_2} (E) = \sigma_{\theta_1} (\sigma_{\theta_2} (E))$$

Rule 2: Commutative Rule

a) This rule states that selections operations are commutative.

$$\sigma_{\theta_1} (\sigma_{\theta_2} (E)) = \sigma_{\theta_2} (\sigma_{\theta_1} (E))$$

b) Theta Join (θ) is commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1 \text{ (θ is in subscript with the join symbol)}$$

However, in the case of theta join, the equivalence rule does not work if the order of attributes is considered. Natural join is a special case of Theta join, and natural join is also commutative.

However, in the case of theta join, the equivalence rule does not work if the order of attributes is considered. Natural join is a special case of Theta join, and natural join is also commutative.

Rule 3: Cascade of Π

This rule states that we only need the final operations in the sequence of the projection operations, and other operations are omitted. Such a transformation is referred to as a **cascade of Π** .

$$\Pi_{L1} (\Pi_{L2} (\dots (\Pi_{Ln} (E)) \dots)) = \Pi_{L1} (E)$$

Rule 4: We can combine the selections with Cartesian products as well as theta joins

Rule 4: We can combine the selections with Cartesian products as well as theta joins

1. $\sigma_{\theta} (E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
2. $\sigma_{\theta_1} (E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

Rule 5: Associative Rule

a) This rule states that natural join operations are associative.

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

b) Theta joins are associative for the following expression:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

In the theta associativity, θ_2 involves the attributes from E_2 and E_3 only. There may be chances of empty conditions, and thereby it concludes that Cartesian Product is also associative.

Rule 6: Distribution of the Selection operation over the Theta join.

Under two following conditions, the selection operation gets distributed over the theta-join operation:

a) When all attributes in the selection condition θ_0 include only attributes of one of the expressions which are being joined.

$$\sigma_{\theta_0} (E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0} (E_1)) \bowtie_{\theta} E_2$$

b) When the selection condition θ_1 involves the attributes of E_1 only, and θ_2 includes the attributes of E_2 only.

$$\sigma_{\theta_1 \wedge \theta_2} (E_1 \bowtie E_2) = (\sigma_{\theta_1} (E_1)) \bowtie (\sigma_{\theta_2} (E_2))$$

Rule 7: Distribution of the projection operation over the theta join.

Under two following conditions, the selection operation gets distributed over the theta-join operation:

a) Assume that the join condition θ includes only in $L_1 \cup L_2$ attributes of E_1 and E_2 . Then, we get the following expression:

$$\pi_{L_1 \cup L_2} (E_1 \bowtie E_2) = (\pi_{L_1} (E_1)) \bowtie (\pi_{L_2} (E_2))$$

b) Assume a join as $E_1 \bowtie E_2$. Both expressions E_1 and E_2 have sets of attributes as L_1 and L_2 . Assume two attributes L_3 and L_4 where L_3 be attributes of the expression E_1 , involved in the θ join condition but not in $L_1 \cup L_2$. Similarly, an L_4 be attributes of the expression E_2 involved only in the θ join condition and not in $L_1 \cup L_2$ attributes. Thus, we get the following expression:

$$\pi_{L_1 \cup L_2} (E_1 \bowtie E_2) = \pi_{L_1 \cup L_2} ((\pi_{L_1 \cup L_3} (E_1)) \bowtie (\pi_{L_2 \cup L_4} (E_2)))$$

Rule 8: The union and intersection set operations are commutative.

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

However, set difference operations are not commutative.

Rule 9: The union and intersection set operations are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Rule 10: Distribution of selection operation on the intersection, union, and set difference operations.

The below expression shows the distribution performed over the set difference operation.

$$\sigma_p (E_1 - E_2) = \sigma_p(E_1) - \sigma_p(E_2)$$

We can similarly distribute the selection operation on $\cup$ and $\cap$ by replacing with $-$. Further, we get:

$$\sigma_p (E_1 - E_2) = \sigma_p(E_1) - E_2$$

Rule 11: Distribution of the projection operation over the union operation.

This rule states that we can distribute the projection operation on the union operation for the given expressions.

$$\pi_L (E1 \cup E2) = (\pi_L (E1)) \cup (\pi_L (E2))$$